Alarm correlation support on SONiC OT

Overview of current alarm management in SONiC OT

SONiC uses the vendor's OTAI library to interact with hardware. During the creation of a linecard object (handled by the Syncd-OT CRUD module), Syncd-OT registers OTAI alarm notification callbacks with the vendor OTAI library. Alarm management in SONiC OTAI is centered around this Notification callback. As soon as the OTAI library detects presence/clear of an alarm hardware event (e.g., loss of signal), it triggers the registered callback.

Alarm Handling Process

Alarm notification mechanism

- The optical linecard or component (e.g., OA, OSC) detects an abnormal condition (e.g., Loss of Signal - LOS).
- The vendor's OTAI library captures the event and triggers the registered callback
- Syncd-OT receives alarm metadata (type, severity, timestamp, etc.).
- This alarm is logged into the `state_db` under the `CURALARM` table.
- Monitoring systems polling the `state_db` can then display or act on this alarm.
- Once the condition is resolved, OTAI again invokes the registered callback.
- Syncd-OT detects this as a cleared state.
- The alarm entry is removed from `CURALARM` table in `state_db`.
- The cleared alarm is then logged in `history_db` under the `HISALARM` table, preserving alarm history.

The above steps can be summarized as below:

```
Linecard Event → OTAI Library → Callback →
    ├── Add to state_db (CURALARM) [if raised]
    └── Remove from state_db, Add to history_db (HISALARM) [if cleared]
```

As outlined above,following tables are used to store alarm related information in redis DB:

- **state_db:CURALARM** – Stores active alarms.
- **history_db:HISALARM** – Stores cleared alarms for long-term tracking.

Limitations of current implementation

No alarm soaking

Alarm soaking is a technique used in network and system monitoring to prevent flapping or transient conditions from generating unnecessary or misleading alarms. Soaking means delaying the activation or clearing of an alarm for a certain amount of time to ensure that the condition is persistent and not just a short-lived glitch.

The alarms on SONiC are handled currently without any soak which means that they are reported to northbound monitoring system as soon as they are detected and similarly clear of alarms happens instantly. This leaves system susceptible to alarm flapping. Micro flaps can lead to constant alarm raise and clear resulting in operator confusion and monitoring system overload.

Also, soaking helps ensure only stable and confirmed alarms are processed for correlation and root cause analysis thereby improving alarm correlation accuracy.

No alarm correlation

A single fault in a large network may result in large number of faults and operator may be overwhelmed by the number of alarms generated in the system and it is very difficult to determine what caused the error. Alarm correlation is a technique used in the network to automatically analyze and interpret alarms generated by various components in a system, with the goal of identifying the root cause of problems and reducing the number of alerts that operators need to handle. The root cause alarm is reported as 'active' to northbound alarm monitoring entity and other alarms that due to the root cause alarms are 'suppressed'. Alarm monitoring agents have capability to read the suppressed alarms as well.

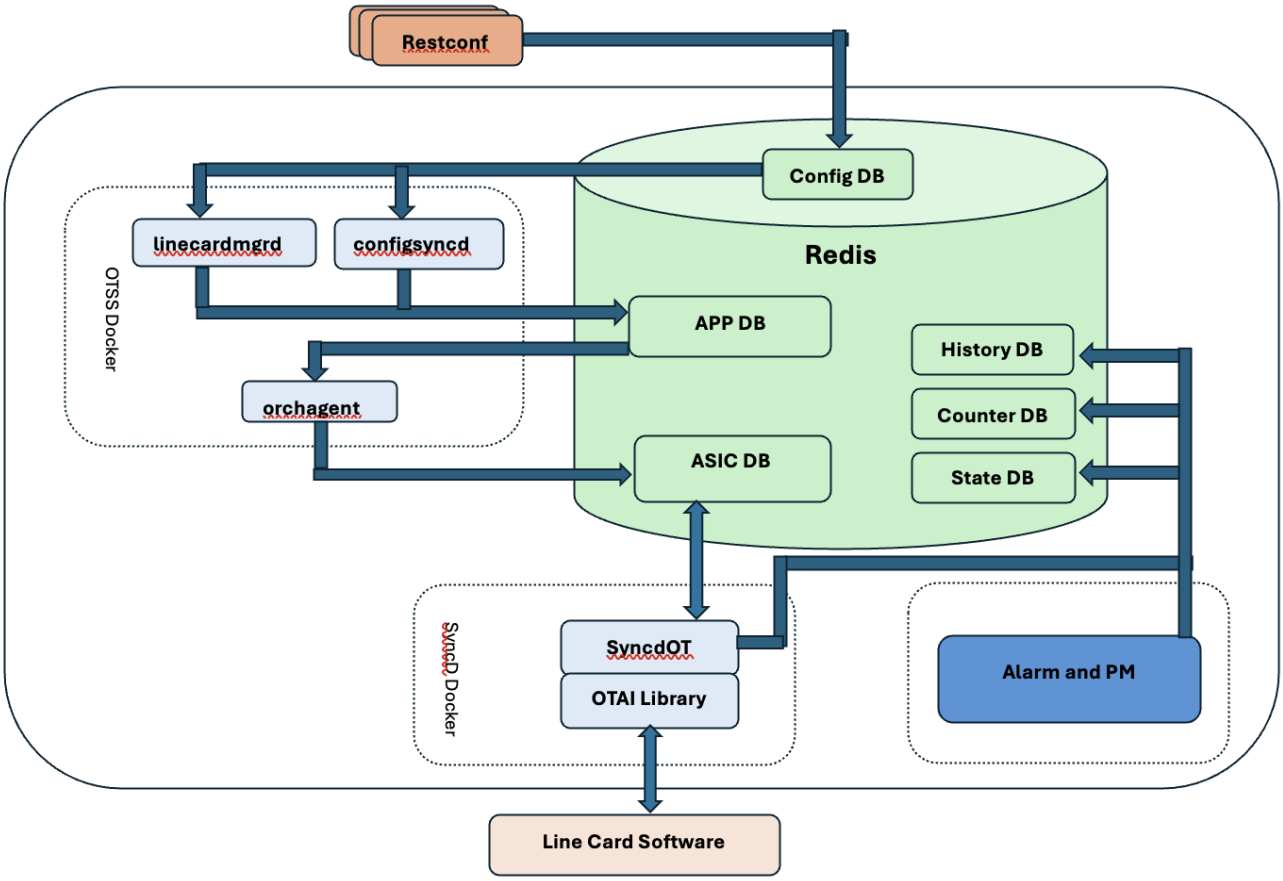
Currently alarm correlation is not supported on SONiC OT.

Scope of this feature

- Correlation will be done on per port basis. Alarms reported on a port will be grouped and correlated for root cause alarm for that port only.
- The alarm suppression will be based only on the static high list and low list. For a given alarm 'A', high list is the list of alarms that will suppress 'A' and low list is the list of alarms that will be suppressed by 'A'. These lists will be static and cannot be changed at runtime.
- Alarm soaking will be based on fixed default values for all alarms and cannot be changed at runtime.
- Alarm profile will be static (alarm severity etc.)
- The static high list/low list (for suppresion), soak time and alarm severity will be defined in a json file that will be read by the Alarm processing engine during initialization.
- Alarms may be reported as 'events' based on severity

High level design

Following the philosophy of SONiC to run each major component in a separate docker container, the new alarm correlation service will have its own container. This will alow to isolate functionalities, reduce interdependency and develop, replace or upgrade the 'alarm correlation' component independently.



The alarm correlation feature is encapsulated as an alarm engine micro service within a new docker container and enables true plug and play. Also, Syncd-OT container and its alarm logic is not affected by the new alarm engine feature. The existing uncorrelated and un-soaked alarm behavior will still be available in state_db and history_db in CURALARM and HISALARM tables.

Alarm engine will use the existing CURALARM table of STATE_DB to synchronize with the current state of alarms in the system.

Alarm engine polls for alarms periodically from STATE_DB: There will be one second poller that reads the alarms from the STATE_DB and then compare the internal alarm state of alarm engine with the DB state to detect new or cleared alarms. The detected changes will be processed accordingly.

Database schema

Correlated alarms will be saved in the existing STATE_DB and HISTORY_DB. However, we are not going to use the CURALARM or HISALARM tables for this feature. The correlated alarms will be stored in new tables:

STATE_DB

- new ACTALARM table for active alarms
- new SUPPALARM table for suppressed alarms

Keys will be ACTALARM|<id> and SUPPALARM|<id> where id is <resource#type-id> Eg: ACTALARM|CH111#GE_RX_LF

Fields:

- id
- time-created
- resource
- text
- severity
- type-id
- service-affecting

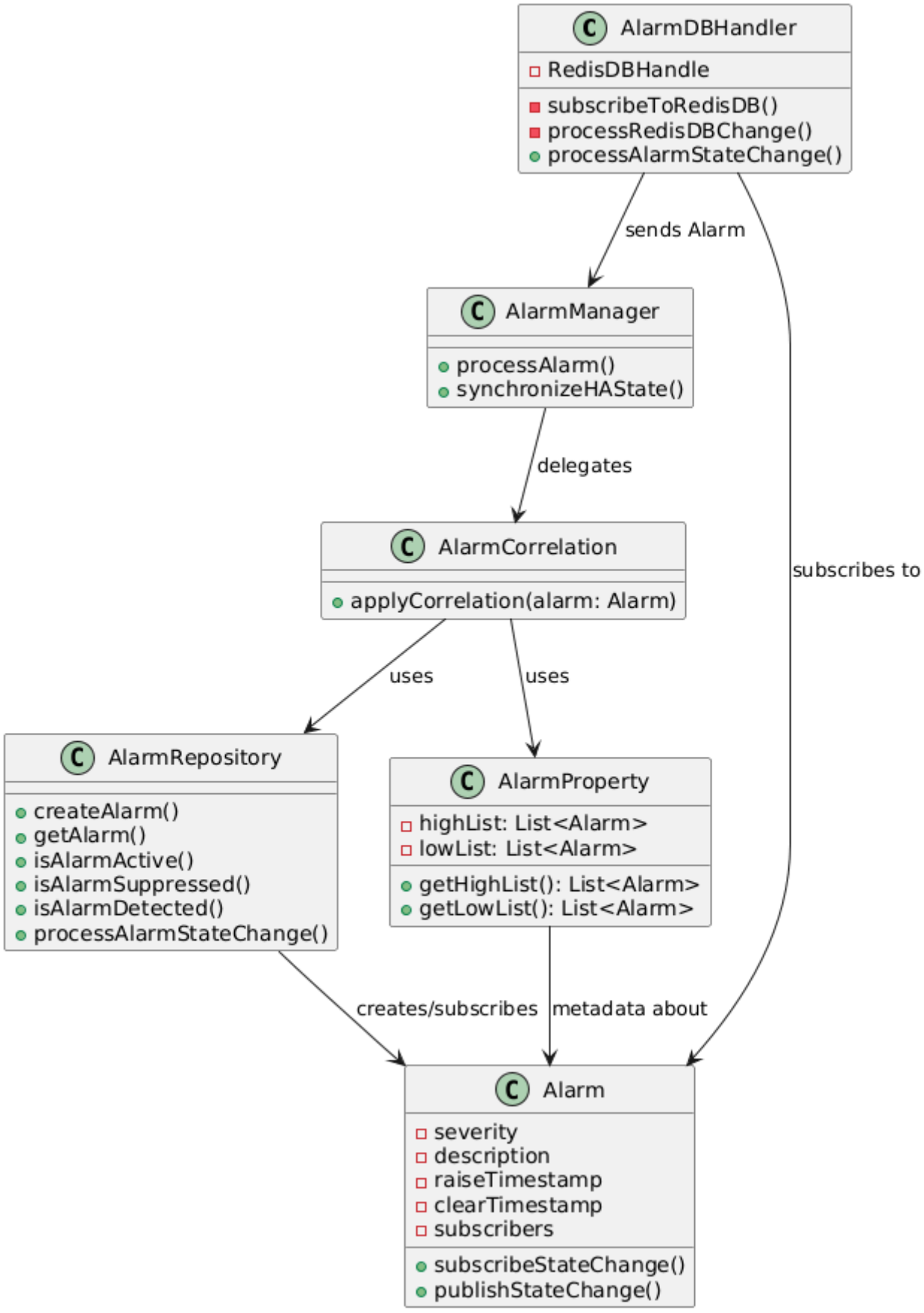
HISTORY_DB

- New CORHISALARM table for correlated historical alarms
- Alarms that are cleared in suppressed state won't be moved to this history table
- Alarms will be kept in history for a durartion of 7 days.

Key will be CORHISALARM|<id>#<time-created> where id is <resource#type-id>

Fields will be same as those in STATE_DB.

Class Diagram



Class: AlarmDBHandler

This class represents the entry/exit point of Alarm engine for all alarms that are raised/cleared on the system. This class is a placeholder and we may want to add new class types in this layer to handle other

responsibilities at the entry/exit point (for example maintaining the relationship between various objects using the CONFIG_DB)

Responsibilities:

- Subscribes to Redis for STATE_DB change related notifications. Hides the Redis DB related details from rest of the module.
- Converts raw DB alarms into Alarm objects via AlarmRepository
- Passes Alarm objects to AlarmManager for further processing.
- Listens to Alarm object state changes and updates Redis DB accordingly (ACTIVE/SUPPRESSED)
- Moves cleared active alarms to the new table CORHISALARM in history DB
- In future, it can support other Redis DB change notifications.

Class: AlarmRepository

Responsibilities:

- Maintains per-port alarm lists:
- All detected alarms
- Active alarms
- Suppressed alarms
- (Future) Soak alarms
- Subscribes to Alarm state changes. This is used to move the alarms to correct list (ACTIVE/SUPPRESSED/SOAK)
- Creates/removes Alarm objects
- Provides interface to check if an Alarm object is in a given state
- Used by: AlarmDBHandler, AlarmManager, AlarmCorrelation

Class: AlarmManager

Responsibilities:

- Receives alarms from AlarmDBHandler
- Processes alarms (future: config-based logic)
- Interfaces with AlarmCorrelation
- Synchronizes alarm state during HA scenarios

Class: Alarm

Responsibilities:

- Holds alarm data: severity, description, timestamps
- Notifies subscribers (AlarmDBHandler, AlarmRepository) on state change
- Created by: AlarmRepository

Class: AlarmCorrelation

Responsibilities:

- Applies correlation logic to alarms

- Uses AlarmRepository and AlarmProperty
- Updates Alarm state based on correlation output

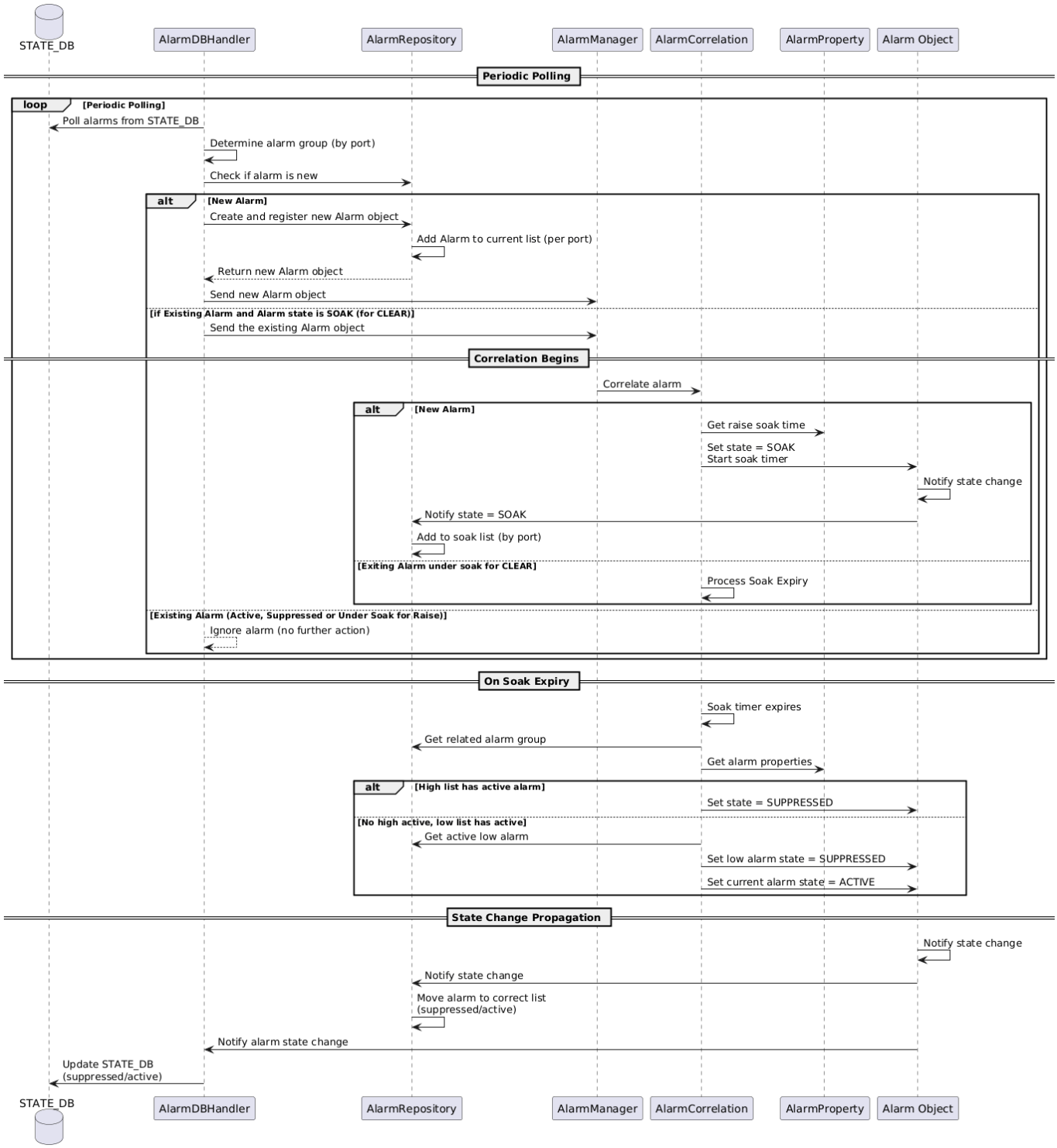
Class: AlarmProperty

Responsibilities:

- Stores metadata for each alarm
- Maintains:
- High list: alarms that suppress it
- Low list: alarms it suppresses

Event Flow

New Alarm processing



1. Polling

- `AlarmDBHandler` periodically polls alarms from `STATE_DB`.

2. Group Resolution

- Based on the alarm source (typically the port), `AlarmDBHandler` identifies the alarm group.

3. Alarm Existence Check

- `AlarmDBHandler` queries `AlarmRepository` to determine if the alarm already exists.

Case A: New Alarm

- `AlarmDBHandler` requests `AlarmRepository` to:
  - Create a new `Alarm` object.
  - Add it to the current alarms list for the port.
- `AlarmDBHandler` forwards the new `Alarm` object to `AlarmManager` for further processing.

Case B: Existing Alarm

- If the existing alarms is under soak, the `Alarm` object is sent to AlarmManager ` for further processing.
- Else, the alarm is **ignored**. No correlation or state update occurs.

4. Initial Correlation

- `AlarmManager` passes the `Alarm` to `AlarmCorrelation`.
- `AlarmCorrelation`:
  - If its an existing alarm under soak, the alarm is processes for correlation as if the soak timer has expired (i.e the processing moves to step 6)
  - Fetches raise soak time using `AlarmProperty`.
  - Sets alarm state to `SOAK`.
  - Starts the soak timer.

5. SOAK State Notification

- The `Alarm` object sends a state change notification.
- `AlarmRepository`:
  - Adds the alarm to the **soak list** for the corresponding port.

6. Soak Expiry and Post-Correlation

- Once the soak timer expires:
  - `AlarmCorrelation` re-evaluates the alarm using correlation logic.
  - It checks related alarms (high/low lists) to decide the next state:

State Transition Logic:

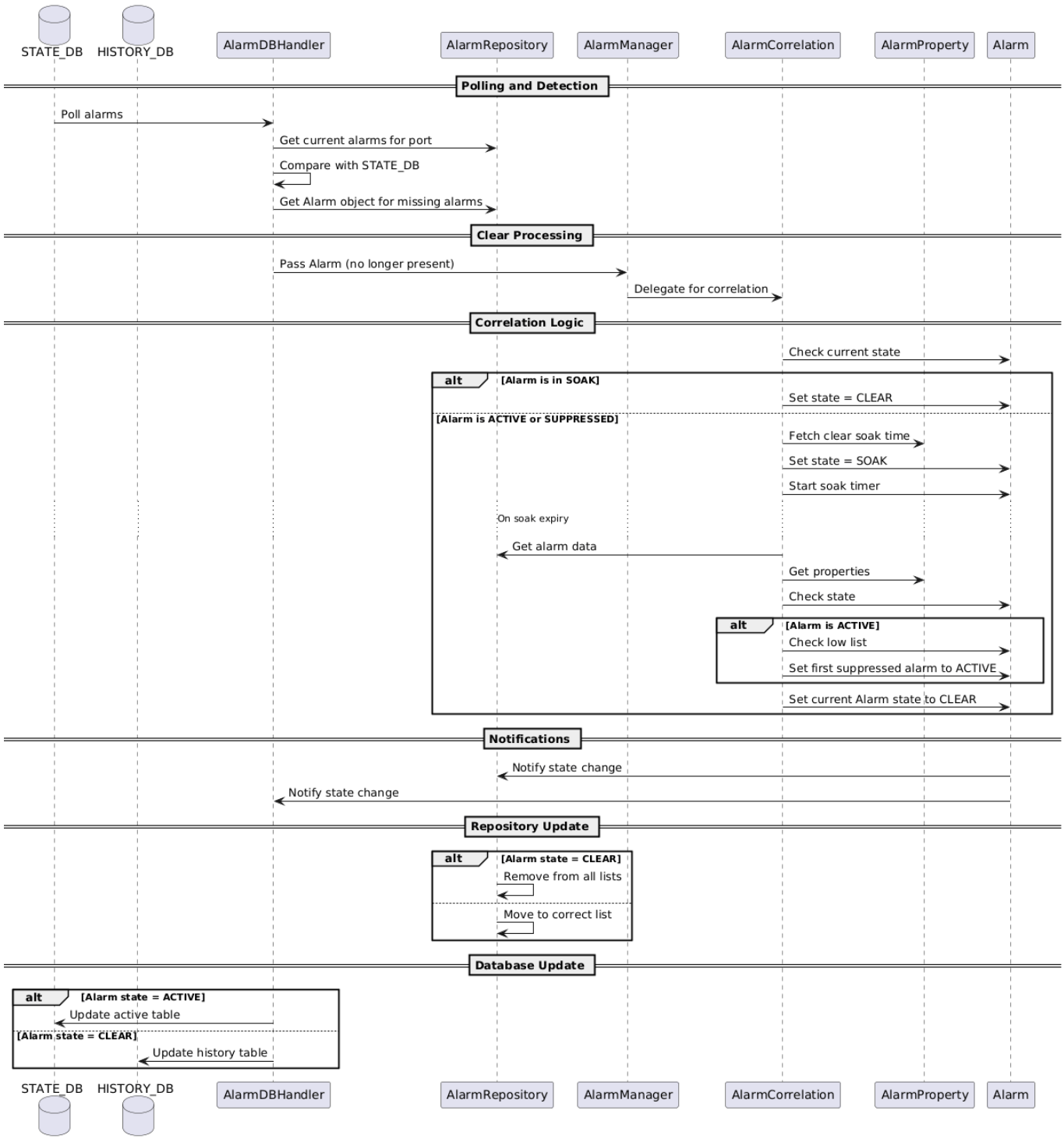
- **If a high alarm is active**:
  - Current alarm → `SUPPRESSED`
- **If no high alarms are active** but a **low alarm is active**:
  - Low alarm → `SUPPRESSED`
  - Current alarm → `ACTIVE`

7. State Change Notification

- Any `Alarm` whose state changes sends a notification to subscribers.
- `AlarmRepository`:
  - Moves the alarm to the appropriate list (`active`, `suppressed`).

- `AlarmDBHandler`:
  - Updates the `STATE_DB` to reflect the new alarm state.

Alarm clear processing



1. Polling from STATE_DB

- `AlarmDBHandler` polls alarms from `STATE_DB`.

2. Detect Cleared Alarms

- `AlarmDBHandler` compares alarms in `STATE_DB` with the current alarm list in `AlarmRepository` for a given port.
- Identifies alarms no longer present in `STATE_DB`.

3. Fetch Cleared Alarm

- For each missing alarm:
  - `AlarmDBHandler` retrieves the `Alarm` object from `AlarmRepository`.

4. Forward for Processing

- `AlarmDBHandler` passes the `Alarm` object to `AlarmManager`, indicating the alarm is no longer present.

5. Correlation Delegation

- `AlarmManager` forwards the `Alarm` object to `AlarmCorrelation`.

6. Correlation Logic

- `AlarmCorrelation` checks the current state of the `Alarm`.

If Alarm is in SOAK:

- Alarm state is updated to `CLEAR`.

Else:

- `AlarmCorrelation` fetches the clear soak time from `AlarmProperty`.
- Updates alarm state to `SOAK` and starts soak timer.

On Soak Expiry:

- Reruns correlation using `AlarmRepository` and `AlarmProperty`.
- If alarm was `ACTIVE`:
  - Activates the first suppressed alarm from the low list.
  - Clears the current alarm.

7. Notifications

- `Alarm` notifies all registered `Subscribers` of state changes.
- `AlarmRepository` receives the notification and updates lists.
- `AlarmDBHandler` receives the notification.

8. Repository Update

If state = CLEAR:

- Removes alarm from all internal lists.

Else:

- Moves alarm to the appropriate state list.

9. Database Update

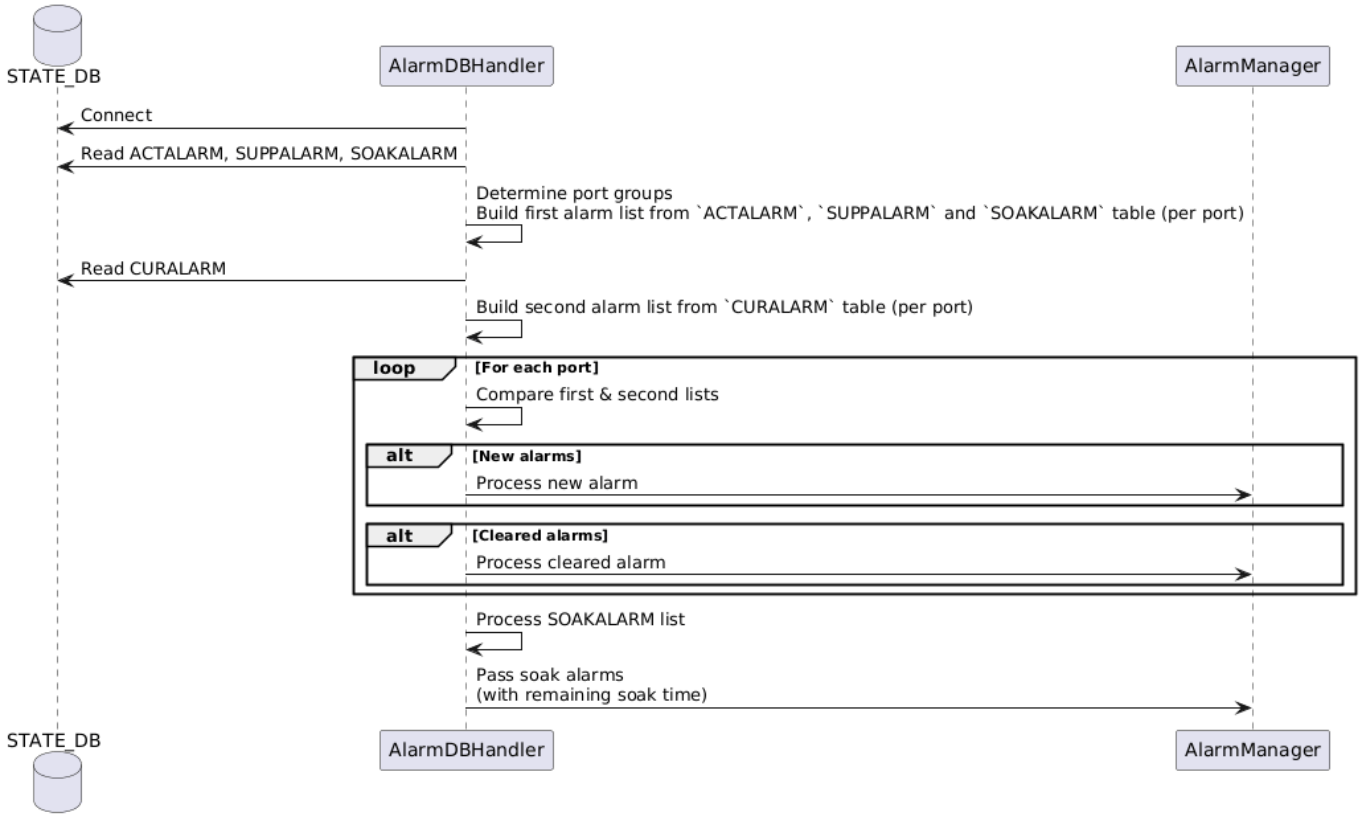
If state = ACTIVE:

- `AlarmDBHandler` updates the `STATE_DB` active table.

If state = CLEAR:

- `AlarmDBHandler` updates the `HISTORY_DB`.

Synchronizing correlated alarm in HA scenarios (Alarm engine container restart, Alarm engine process restart, Controller card failover to standby)



1. Connect to STATE_DB

`AlarmDBHandler` establishes a connection with `STATE_DB`.

2. Read Active Alarm Tables

It reads all alarms from the following tables in `STATE_DB`:

- `ACTALARM`: Active alarms
- `SUPPALARM`: Suppressed alarms

- `SOAKALARM`: Alarms currently under soak

> **Note:** The `SOAKALARM` table includes the remaining soak time (in seconds) for each alarm.

3. Build Alarm List from `ACTALARM`, `SUPPALARM` and `SOAKALARM` table(Reference List)

For each alarm retrieved, `AlarmDBHandler`:

- Identifies the port group the alarm belongs to
- Builds the first list of **current alarms per port**

4. Read Current Alarms from `CURALARM` table

`AlarmDBHandler` reads all alarms from the `CURALARM` table and builds a second list of **current alarms per port**.

5. Compare Alarm Lists

For each port, the manager compares:

- The first list (from `ACTALARM`, `SUPPALARM`, `SOAKALARM`)
- The second list (from `CURALARM`)

6. Process New Alarms

If an alarm is found in the second list but not in the first, it is considered a **new alarm**.

- The new alarm is sent to `AlarmManager` for processing.
- Refer to the event flow for new alarms.

7. Process Cleared Alarms

If an alarm is in the first list but not in the second, it is considered a **cleared alarm**.

- The cleared alarm is sent to `AlarmManager` for processing.
- Refer to the event flow for cleared alarms.

8. Process Soak Alarms

`AlarmDBHandler` processes alarms listed in the `SOAKALARM` table separately.

9. Delegate Soak Processing

Alarms under soak are passed to `AlarmManager`:

- Each alarm includes the remaining soak time
- `AlarmManager` handles further soak-time processing